

Introduction to Electronic Design Automation

Homework 2

(Problems 6-8)

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

6 Two-level Logic Minimization

6. (a)

Use variable order $abcde$.

$$G = abce + abde + ab\bar{c}e + \bar{a}b\bar{c}e + \bar{a}\bar{b}\bar{c}\bar{e} + \bar{a}\bar{c}\bar{d}e + bc\bar{d}e + cd\bar{e}$$

$$D = \bar{a}b\bar{d}e + abcd\bar{e} + ac\bar{e}$$

Expanding them into minterms:

$$G = \{0, 1, 2, 4, 9, 11, 12, 13, 20, 25, 27, 28, 29, 31\}$$

$$D = \{9, 13, 20, 22, 28, 29, 30\}$$

Thus, we have the minterms of onset F as:

$$F = G - D = \{0, 1, 2, 4, 11, 12, 25, 27, 31\}$$

Therefore,

$$F \cup D = \{0, 1, 2, 4, 9, 11, 12, 13, 20, 22, 25, 27, 28, 29, 30, 31\}$$

$$\begin{aligned} R &= \{0, 1, \dots, 31\} - (F \cup D) \\ &= \{3, 5, 6, 7, 8, 10, 14, 15, 16, 17, 18, 19, 21, 23, 24, 26\} \end{aligned}$$

Writing them as product terms:

$$\begin{aligned} R &= \bar{a}\bar{b}\bar{c}de + \bar{a}\bar{b}c\bar{d}e + \bar{a}\bar{b}cd\bar{e} + \bar{a}\bar{b}cde + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}b\bar{c}d\bar{e} \\ &\quad + \bar{a}b\bar{c}d\bar{e} + \bar{a}b\bar{c}de + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}b\bar{c}d\bar{e} + \bar{a}b\bar{c}de + \bar{a}b\bar{c}d\bar{e} \\ &\quad + \bar{a}b\bar{c}de + \bar{a}b\bar{c}d\bar{e} + \bar{a}b\bar{c}de + \bar{a}b\bar{c}d\bar{e} \end{aligned}$$

6. (b)

First, we apply the Quine–McCluskey tabular method on $F \cup D$ to generate all prime implicants.

Round 1	Round 2	Round 3
$\bar{a}\bar{b}\bar{c}\bar{d}\bar{e} \checkmark$	$\bar{a}\bar{b}\bar{c}\bar{d}$ $\bar{a}\bar{b}\bar{c}\bar{e}$ $\bar{a}\bar{b}\bar{d}\bar{e}$	
$\bar{a}\bar{b}\bar{c}d\bar{e} \checkmark$ $\bar{a}\bar{b}c\bar{d}\bar{e} \checkmark$ $\bar{a}b\bar{c}\bar{d}\bar{e} \checkmark$	$\bar{a}\bar{c}\bar{d}\bar{e}$ $\bar{b}c\bar{d}\bar{e} \checkmark$ $\bar{a}c\bar{d}\bar{e} \checkmark$	
$\bar{a}\bar{b}\bar{c}d\bar{e} \checkmark$ $\bar{a}b\bar{c}\bar{d}\bar{e} \checkmark$ $a\bar{b}\bar{c}\bar{d}\bar{e} \checkmark$	$\bar{a}\bar{b}\bar{c}e \checkmark$ $\bar{a}bde \checkmark$ $\bar{b}\bar{c}\bar{d}e \checkmark$ $\bar{a}bcd \checkmark$ $\bar{b}c\bar{d}\bar{e} \checkmark$ $\bar{a}b\bar{c}e \checkmark$ $ac\bar{d}\bar{e} \checkmark$	$c\bar{d}\bar{e}$ $b\bar{d}e$ $b\bar{c}e$ bcd $ac\bar{e}$
$\bar{a}\bar{b}\bar{c}de \checkmark$ $\bar{a}b\bar{c}d\bar{e} \checkmark$ $a\bar{b}\bar{c}d\bar{e} \checkmark$ $a\bar{b}c\bar{d}\bar{e} \checkmark$ $abc\bar{d}\bar{e} \checkmark$	$\bar{b}\bar{c}de \checkmark$ $\bar{b}c\bar{d}e \checkmark$ $ac\bar{e} \checkmark$ $ab\bar{c}e \checkmark$ $ab\bar{d}e \checkmark$ $abcd \checkmark$ $abc\bar{e} \checkmark$	abe abc
$ab\bar{c}d\bar{e} \checkmark$ $abc\bar{d}\bar{e} \checkmark$ $abcd\bar{e} \checkmark$	$abde \checkmark$ $abce \checkmark$ $abcd \checkmark$	
$abcde \checkmark$		

Table 1: Quine–McCluskey tabular method for generating prime implicants

Thus, the prime implicants are

$$\{\bar{a}\bar{b}\bar{c}\bar{d}, \bar{a}\bar{b}\bar{c}\bar{e}, \bar{a}\bar{b}\bar{d}\bar{e}, \bar{a}\bar{c}\bar{d}\bar{e}, c\bar{d}\bar{e}, b\bar{d}e, \bar{b}\bar{c}e, bcd, ac\bar{e}, abe, abc\}.$$

Next, we use the Column covering of Boolean matrix to find the essential prime implicants. The Boolean matrix is as follows:

	$\bar{a}\bar{b}\bar{c}\bar{d}$	$\bar{a}\bar{b}\bar{c}\bar{e}$	$\bar{a}\bar{b}\bar{d}\bar{e}$	$\bar{a}\bar{c}\bar{d}\bar{e}$	$c\bar{d}\bar{e}$	$b\bar{d}e$	$\bar{b}\bar{c}e$	bcd	$ac\bar{e}$	abe	abc
$m_0 = \bar{a}\bar{b}\bar{c}\bar{d}\bar{e}$	1	1	1	0	0	0	0	0	0	0	0
$m_1 = \bar{a}\bar{b}\bar{c}d\bar{e}$	1	0	0	1	0	0	0	0	0	0	0
$m_2 = \bar{a}\bar{b}c\bar{d}\bar{e}$	0	1	0	0	0	0	0	0	0	0	0
$m_4 = \bar{a}\bar{b}cd\bar{e}$	0	0	1	0	1	0	0	0	0	0	0
$m_{11} = \bar{a}b\bar{c}d\bar{e}$	0	0	0	0	0	0	1	0	0	0	0
$m_{12} = \bar{a}bcd\bar{e}$	0	0	0	0	1	0	0	1	0	0	0
$m_{25} = ab\bar{c}\bar{d}\bar{e}$	0	0	0	0	0	1	1	0	0	1	0
$m_{27} = ab\bar{c}d\bar{e}$	0	0	0	0	0	0	1	0	0	1	0
$m_{31} = abcde$	0	0	0	0	0	0	0	0	0	1	1

Table 2: Boolean matrix

6. (c)

Rows m_2 and m_{11} each have only one 1, so $\bar{a}\bar{b}\bar{c}\bar{e}$ and $b\bar{c}e$ are essential prime implicants. So we remove the rows m_2, m_{11} and the columns of $\bar{a}\bar{b}\bar{c}\bar{e}$ and $b\bar{c}e$. We also remove the rows that have 1 in the removed columns, which are m_0, m_{25}, m_{27} . After removing these rows, the columns $a\bar{c}\bar{e}$ and $b\bar{d}e$ contain only zeros, so we remove them as well. The resulting Boolean matrix is as follows:

	$\bar{a}\bar{b}\bar{c}\bar{d}$	$\bar{a}\bar{b}\bar{d}\bar{e}$	$\bar{a}\bar{c}\bar{d}e$	$c\bar{d}\bar{e}$	bcd	abe	abc
$m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$	1	0	1	0	0	0	0
$m_4 = \bar{a}\bar{b}c\bar{d}\bar{e}$	0	1	0	1	0	0	0
$m_{12} = \bar{a}bc\bar{d}\bar{e}$	0	0	0	1	1	0	0
$m_{31} = abcde$	0	0	0	0	0	1	1

Table 3: Reduced Boolean matrix

6. (d)

Column $c\bar{d}\bar{e}$ covers rows m_4 and m_{12} . It dominates both columns $\bar{a}\bar{b}\bar{d}\bar{e}$ and bcd , so we remove these two dominated columns. The resulting Boolean matrix is as follows:

	$\bar{a}\bar{b}\bar{c}\bar{d}$	$\bar{a}\bar{c}\bar{d}e$	$c\bar{d}\bar{e}$	abe	abc
$m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$	1	1	0	0	0
$m_4 = \bar{a}\bar{b}c\bar{d}\bar{e}$	0	0	1	0	0
$m_{12} = \bar{a}bc\bar{d}\bar{e}$	0	0	1	0	0
$m_{31} = abcde$	0	0	0	1	1

Table 4: Reduced Boolean matrix

Since rows m_4 and m_{12} have only one 1, the prime implicant $c\bar{d}\bar{e}$ is an induced essential prime implicant. Therefore, we select it and remove rows m_4 and m_{12} .

	$\bar{a}\bar{b}\bar{c}\bar{d}$	$\bar{a}\bar{c}\bar{d}e$	abe	abc
$m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$	1	1	0	0
$m_{31} = abcde$	0	0	1	1

Table 5: Reduced Boolean matrix

6. (e)

After the reductions in (d), there is no cyclic core to solve, since the remaining matrix is decomposed into two independent single-row choices:

$$m_1 : \{\bar{a}\bar{b}\bar{c}\bar{d}, \bar{a}\bar{c}\bar{d}e\}$$

$$m_{31} : \{abe, abc\}$$

Thus, the independent set heuristic is not applicable. We only need to choose one prime implicant from each set. For example, we choose

$$\bar{a}\bar{b}\bar{c}\bar{d} \quad \text{and} \quad abe.$$

Therefore, the selected prime implicants are

$$\bar{a}\bar{b}\bar{c}\bar{e}, \quad b\bar{c}e, \quad c\bar{d}\bar{e}, \quad \bar{a}\bar{b}\bar{c}\bar{d}, \quad abe.$$

6. (f)

There are four minimum covers with 5 prime implicants:

$$G_1^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{b}\bar{c}\bar{d} + abe.$$

$$G_2^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{c}\bar{d}e + abe.$$

$$G_3^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{b}\bar{c}\bar{d} + abc.$$

$$G_4^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{c}\bar{d}e + abc.$$

7 Static Timing Analysis**7. (a)**

In this part, we construct the circuit as a DAG, where each node $v \in V$ represents a gate and each edge $e \in E \subseteq V \times V$ represents a connection between gates. Let $R(v)$ denote the required time at the input of node v , $A(v)$ denote the arrival time at the output of node v , and $d_{u,v}$ denotes the delay from node u 's output to node v 's output. Also, we define the slack of node v as

$$S(v) = R(v) - A(v).$$

Suppose there is a node v such that

$$S(v) = R(v) - A(v) \leq c$$

We want to show that there exists a path satisfying every node on the path has slack at most c from some primary input to v and from v to some primary output.

1. From some primary input to v

First, if v is a primary input, then we are done. Otherwise, since

$$A(v) = \max_{u:(u,v) \in E} (A(u) + d_{u,v})$$

there exist a node u such that there is an edge from u to v and

$$A(v) = A(u) + d_{u,v}.$$

And since

$$R(u) = \min_{w:(u,w) \in E} (R(w) - d_{u,w}) \leq R(v) - d_{u,v},$$

we have

$$S(u) = R(u) - A(u) \leq R(v) - d_{u,v} - A(u) = R(v) - A(v) = S(v) \leq c.$$

Thus, we can apply the same argument on u and repeat this process until we reach a primary input.

2. From v to some primary output

First, if v is a primary output, then we are done. Otherwise, since

$$R(v) = \min_{w:(v,w) \in E} (R(w) - d_{v,w})$$

there exist a node w such that there is an edge from v to w and

$$R(v) = R(w) - d_{v,w}.$$

And since

$$A(w) = \max_{x:(x,w) \in E} (A(x) + d_{x,w}) \geq A(v) + d_{v,w},$$

we have

$$S(w) = R(w) - A(w) \leq R(v) + d_{v,w} - A(v) - d_{v,w} = R(v) - A(v) = S(v) \leq c.$$

Thus, we can apply the same argument on w and repeat this process until we reach a primary output.

Hence, there exists a path from some primary input to v and from v to some primary output such that the slack of each node on these paths is at most c .

7. (b)

The equivalence DAG of the given circuit is as Figure 1 below. The delay of each gate is labeled on the corresponding node. Arrival time, required time and slack of each node are also labeled on the corresponding node in the format of (a, r, s) , where a is the arrival time, r is the required time and s is the slack. The critical path is highlighted in red.

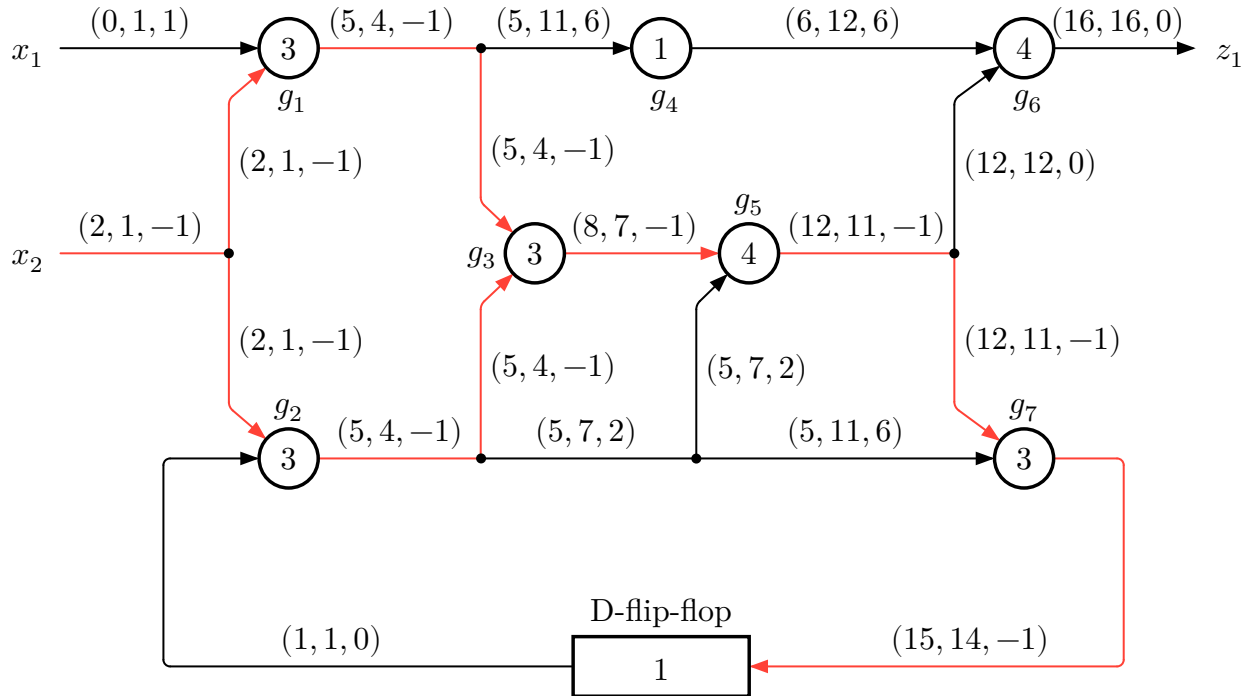


Figure 1: static timing analysis result

8 Technology Mapping

In this problem, I wrote two versions of the program. The first one is the basic DAGON mapping algorithm. The second one is an improved version, where I insert two inverters on every original edge before running DAGON. This gives the mapper more chance to use `INV` and `buf`.

The node numbering I used in my input file is shown in Figure 2.

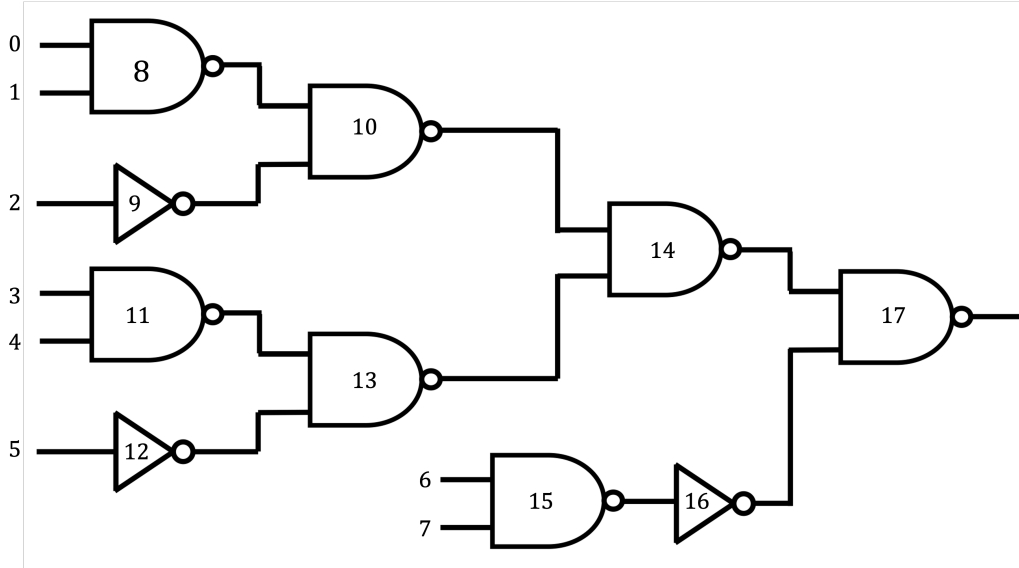


Figure 2: Node numbering used in `input.txt`

First, for the basic version without improvement, the result is:

Node	Minimum cost	Selected cell
8	2	nand2
9	1	inv
10	5	nand2
11	2	nand2
12	1	inv
13	5	nand2
14	12	nand2
15	2	nand2
16	3	inv
17	15	nand3

Table 6: Optimal cost and selected cell of each subject node

Thus, the final cover without improvement is obtained by tracing back from output node 17:

$\{8 : \text{nand2}, 9 : \text{inv}, 10 : \text{nand2}, 11 : \text{nand2}, 12 : \text{inv}, 13 : \text{nand2}, 14 : \text{nand2}, 17 : \text{nand3}\}$.

The `nand3` at node 17 covers nodes 15, 16, and 17 together. Therefore, the minimum total area is

$$2 + 1 + 2 + 2 + 1 + 2 + 2 + 3 = 15.$$

Next, for the improved version, the output on original nodes is:

Node	Minimum cost	Selected cell
8	2	nand2
9	1	inv
10	5	nand2
11	2	nand2
12	1	inv
13	5	nand2
14	10	nand2
15	2	nand2
16	3	inv
17	12	oai21

Table 7: Improved result on original subject nodes

The minimum total area becomes 12. This is smaller than 15 because the inserted inverter pairs create extra matching choices. In particular, node 17 can be matched by **oai21** in the improved graph, so the final cost is reduced. However, if we only apply the original DAGON algorithm directly on the given subject graph, the answer is still 15.

8. (a) Appendix: dagon.cpp

cpp

```

1  #pragma GCC optimize("Ofast")
2  #include <bits/stdc++.h>
3  using namespace std;
4  #define ll long long
5  #define pb push_back
6  #define endl '\n'
7  #define AI(x) begin(x),end(x)
8  #ifdef DEBUG
9  #define debug(args...) LKJ("\033[1;32m[ "+string(#args)+" ]\033[0m", args)
10 template<class I> void LKJ(I&&x){ cerr << x << '\n'; }
11 template<class I, class...T> void LKJ(I&&x, T&&...t){ cerr << x << ", ", LKJ(t...); }
12 template<class I> void OI(I a, I b){ while(a < b) cerr << *a << " \n"[next(a) == b], ++a; }
13 #else
14 #define debug(...) 0
15 #define OI(...) 0
16 #endif
17 #define _ ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
18
19 const int INF = 1e9;
20
21 enum class GateType {
22     PI,
23     INV,
24     NAND,
25     NOR,
26 };
27
28 string toUpper(string s) {
29     for (char &c : s) c = toupper(c);
30     return s;
31 }
32
33 GateType s2gate_type(string s) {
34     s = toUpper(s);
35
36     if (s == "PI" || s == "IN") return GateType::PI;
37     if (s == "INV") return GateType::INV;
38     if (s == "NAND") return GateType::NAND;
39     if (s == "NOR") return GateType::NOR;
40
41     cerr << "Unknown gate type: " << s << endl;
42     exit(1);
43 }
44
45 bool isCommutative(GateType t) {
46     return t == GateType::NAND || t == GateType::NOR;
47 }
48
49 struct node {
50     int id;
51     GateType type;
52     vector<int> fanin;
53     vector<int> fanout;
54
55     node() {}
56     node(int i, GateType g) : id(i), type(g) {}
57 };
58
59 vector<int> topological_sort(vector<node>& g) {
60     int n = g.size();
61     vector<int> indeg(n), order;
62     queue<int> q;
63
64     for (int i = 0; i < n; i++) {
65         indeg[i] = g[i].fanin.size();
66         if (indeg[i] == 0) q.push(i);
67     }
68
69     while (!q.empty()) {
70         int u = q.front();
71         q.pop();
72
73         order.pb(u);
74
75         for (int v : g[u].fanout) {
76             indeg[v]--;
77             if (indeg[v] == 0) q.push(v);
78         }
79     }
80
81     return order;
82 }
83
84 // 回傳 pattern subtree match 到 subject subtree 後, boundary dp cost 總和
85 // 若不能 match, 回傳 INF
86 int matchCost(int sid, int pid,
87     vector<node>& sg,

```

```

88     vector<node>& pg,
89     vector<int>& dp) {
90
91     node& s = sg[sid];
92     node& p = pg[pid];
93
94     if (p.type == GateType::PI) {
95         return dp[sid];
96     }
97
98     if (s.type != p.type) return INF;
99     if (s.fanin.size() != p.fanin.size()) return INF;
100
101     int n = s.fanin.size();
102
103     vector<int> idx(n);
104     iota(idx.begin(), idx.end(), 0);
105
106     int best = INF;
107
108     do {
109         int total = 0;
110         bool ok = true;
111
112         for (int i = 0; i < n; i++) {
113             int s_child = s.fanin[idx[i]];
114             int p_child = p.fanin[i];
115
116             int sub = matchCost(s_child, p_child, sg, pg, dp);
117
118             if (sub >= INF) {
119                 ok = false;
120                 break;
121             }
122
123             total += sub;
124         }
125
126         if (ok) best = min(best, total);
127     } while (isCommutative(s.type) &&
128             next_permutation(idx.begin(), idx.end()));
129
130     return best;
131 }
132 }
133
134 signed main() {
135     int ns, ms;
136     cin >> ns >> ms;
137
138     vector<node> sg(ns);
139
140     for (int i = 0; i < ns; i++) {
141         string t;
142         cin >> t;
143         sg[i] = node(i, s2gate_type(t));
144     }
145
146     for (int i = 0; i < ms; i++) {
147         int a, b;
148         cin >> a >> b;
149
150         sg[a].fanout.pb(b);
151         sg[b].fanin.pb(a);
152     }
153
154     vector<int> subject_order = topological_sort(sg);
155
156     int p;
157     cin >> p;
158
159     vector<vector<node>> patterns(p);
160     vector<int> cost(p);
161     vector<string> name(p);
162     vector<int> root(p);
163
164     for (int i = 0; i < p; i++) {
165         cin >> name[i] >> cost[i];
166
167         int n, m, r;
168         cin >> n >> m >> r;
169
170         root[i] = r;
171         patterns[i].resize(n);
172
173         for (int j = 0; j < n; j++) {
174             string t;
175             cin >> t;
176             patterns[i][j] = node(j, s2gate_type(t));
177         }

```

```

178         for (int j = 0; j < m; j++) {
179             int a, b;
180             cin >> a >> b;
181
182             patterns[i][a].fanout.pb(b);
183             patterns[i][b].fanin.pb(a);
184         }
185     }
186 }
187
188 vector<int> dp(ns, INF);
189 vector<int> choose(ns, -1);
190
191 for (int v : subject_order) {
192     if (sg[v].type == GateType::PI) {
193         dp[v] = 0;
194         continue;
195     }
196
197     for (int i = 0; i < p; i++) {
198         int boundary_cost = matchCost(v, root[i], sg, patterns[i], dp);
199
200         if (boundary_cost >= INF) continue;
201
202         int total_cost = cost[i] + boundary_cost;
203
204         if (total_cost < dp[v]) {
205             dp[v] = total_cost;
206             choose[v] = i;
207         }
208     }
209 }
210
211 // 預設所有 fanout empty 的 node 都是 output
212 int ans = 0;
213
214 for (int i = 0; i < ns; i++) {
215     if (sg[i].fanout.empty()) {
216         ans += dp[i];
217     }
218 }
219
220 cout << "Minimum total cost = " << ans << endl;
221
222 for (int i = 0; i < ns; i++) {
223     cout << "node " << i << ": ";
224
225     if (sg[i].fanin.empty()) {
226         cout << "PI, cost = 0" << endl;
227     } else {
228         cout << "cost = " << dp[i];
229
230         if (choose[i] != -1) {
231             cout << ", cell = " << name[choose[i]];
232         } else {
233             cout << ", cell = NONE";
234         }
235
236         cout << endl;
237     }
238 }
239
240 return 0;
241 }
242

```

8. (b) Appendix: dagon_improve.cpp

cpp

```

1 // dagon_improve.cpp
2 #pragma GCC optimize("Ofast")
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 #define ll long long
7 #define pb push_back
8 #define endl '\n'
9 #define _ ios::sync_with_stdio(false); cin.tie(nullptr);
10
11 const ll INF = (ll)4e18;
12
13 enum class GateType {
14     PI,
15     INV,
16     NAND,
17     NOR,
18 };
19
20 string toUpper(string s) {
21     for (char &c : s) c = toupper(c);
22     return s;
23 }
24
25 GateType s2gate_type(string s) {
26     s = toUpper(s);
27
28     if (s == "PI" || s == "IN") return GateType::PI;
29     if (s == "INV") return GateType::INV;
30     if (s == "NAND") return GateType::NAND;
31     if (s == "NOR") return GateType::NOR;
32
33     cerr << "Unknown gate type: " << s << endl;
34     exit(1);
35 }
36
37 string gate2str(GateType t) {
38     if (t == GateType::PI) return "PI";
39     if (t == GateType::INV) return "INV";
40     if (t == GateType::NAND) return "NAND";
41     if (t == GateType::NOR) return "NOR";
42     return "?";
43 }
44
45 bool isCommutative(GateType t) {
46     return t == GateType::NAND || t == GateType::NOR;
47 }
48
49 struct Node {
50     int id;
51     GateType type;
52     vector<int> fanin;
53     vector<int> fanout;
54
55     Node() {}
56     Node(int i, GateType g) : id(i), type(g) {}
57 };
58
59 struct Pattern {
60     string name;
61     ll cost;
62     int root;
63     vector<Node> g;
64 };
65
66 void addEdge(vector<Node> &g, int u, int v) {
67     g[u].fanout.pb(v);
68     g[v].fanin.pb(u);
69 }
70
71 vector<int> topological_sort(vector<Node> &g) {
72     int n = (int)g.size();
73     vector<int> indeg(n, 0);
74     queue<int> q;
75
76     for (int i = 0; i < n; i++) {
77         indeg[i] = (int)g[i].fanin.size();
78         if (indeg[i] == 0) q.push(i);
79     }
80
81     while (!q.empty()) {
82         int u = q.front();
83         q.pop();
84
85         order.pb(u);
86
87         for (int v : g[u].fanout) {

```

```

88     if (--indeg[v] == 0) q.push(v);
89     }
90 }
91
92 if ((int)order.size() != n) {
93     cerr << "Error: graph is not a DAG." << endl;
94     exit(1);
95 }
96
97 return order;
98 }
99
100 // 對 subject graph 每條原始 edge u -> v 插入兩個 INV:
101 // u -> inv1 -> inv2 -> v
102 void insertInverterPairs(vector<Node> &sg,
103     const vector<pair<int,int>> &edges,
104     vector<char> &isInserted) {
105     for (auto &x : sg) {
106         x.fanin.clear();
107         x.fanout.clear();
108     }
109
110     for (auto [u, v] : edges) {
111         int inv1 = (int)sg.size();
112         sg.pb(Node(inv1, GateType::INV));
113         isInserted.pb(true);
114
115         int inv2 = (int)sg.size();
116         sg.pb(Node(inv2, GateType::INV));
117         isInserted.pb(true);
118
119         addEdge(sg, u, inv1);
120         addEdge(sg, inv1, inv2);
121         addEdge(sg, inv2, v);
122     }
123 }
124
125 // 回傳 pattern subtree match 到 subject subtree 後的 boundary dp cost 總和。
126 // 若不能 match, 回傳 INF.
127 ll matchCost(int sid, int pid,
128     vector<Node> &sg,
129     vector<Node> &pg,
130     vector<ll> &dp,
131     vector<char> &isInserted) {
132
133     Node &s = sg[sid];
134     Node &pnode = pg[pid];
135
136     // pattern 的 PI 是 boundary, 可以停在任何 subject node
137     if (pnode.type == GateType::PI) {
138         return dp[sid];
139     }
140
141     ll bestSkip = INF;
142
143     // 嘗試跳過 virtual INV pair:
144     // u -> inv1 -> inv2
145     // sid = inv2 時, 可以直接嘗試 match u
146     if (isInserted[sid] &&
147         s.type == GateType::INV &&
148         s.fanin.size() == 1) {
149
150         int inv1 = s.fanin[0];
151
152         if (isInserted[inv1] &&
153             sg[inv1].type == GateType::INV &&
154             sg[inv1].fanin.size() == 1) {
155
156             int u = sg[inv1].fanin[0];
157
158             bestSkip = matchCost(u, pid, sg, pg, dp, isInserted);
159         }
160     }
161
162     if (s.type != pnode.type) return bestSkip;
163     if (s.fanin.size() != pnode.fanin.size()) return bestSkip;
164
165     int n = (int)s.fanin.size();
166
167     vector<int> idx(n);
168     iota(idx.begin(), idx.end(), 0);
169
170     ll best = INF;
171
172     do {
173         ll total = 0;
174         bool ok = true;
175
176         for (int i = 0; i < n; i++) {
177             int s_child = s.fanin[idx[i]];

```

```

178     int p_child = pnode.fanin[i];
179
180     ll sub = matchCost(s_child, p_child, sg, pg, dp, isInserted);
181
182     if (sub >= INF) {
183         ok = false;
184         break;
185     }
186
187     total += sub;
188
189     if (total >= INF) {
190         ok = false;
191         break;
192     }
193 }
194
195 if (ok) best = min(best, total);
196
197 } while (isCommutative(s.type) &&
198         next_permutation(idx.begin(), idx.end()));
199
200 return min(best, bestSkip);
201 }
202
203 int main() {
204     int ns, ms;
205     cin >> ns >> ms;
206
207     vector<Node> sg(ns);
208
209     for (int i = 0; i < ns; i++) {
210         string t;
211         cin >> t;
212         sg[i] = Node(i, s2gate_type(t));
213     }
214
215     vector<pair<int,int>> original_edges;
216
217     for (int i = 0; i < ms; i++) {
218         int a, b;
219         cin >> a >> b;
220         original_edges.pb({a, b});
221         addEdge(sg, a, b);
222     }
223
224     // 記錄原始 output, 因為後面會清掉並重建 edge
225     vector<int> original_outputs;
226     for (int i = 0; i < ns; i++) {
227         if (sg[i].fanout.empty()) original_outputs.pb(i);
228     }
229
230     // improve DAGON: 每條 wire 插入兩個 INV
231     vector<char> isInserted(ns, false);
232     insertInverterPairs(sg, original_edges, isInserted);
233
234     int p;
235     cin >> p;
236
237     vector<Pattern> patterns;
238
239     for (int i = 0; i < p; i++) {
240         Pattern pat;
241         cin >> pat.name >> pat.cost;
242
243         int n, m, r;
244         cin >> n >> m >> r;
245
246         pat.root = r;
247         pat.g.resize(n);
248
249         for (int j = 0; j < n; j++) {
250             string t;
251             cin >> t;
252             pat.g[j] = Node(j, s2gate_type(t));
253         }
254
255         for (int j = 0; j < m; j++) {
256             int a, b;
257             cin >> a >> b;
258             addEdge(pat.g, a, b);
259         }
260
261         patterns.pb(pat);
262     }
263
264     vector<int> order = topological_sort(sg);
265
266     int nsg = (int)sg.size();
267     vector<ll> dp(nsg, INF);

```

```

268     vector<int> choose(nsg, -1);
269
270     for (int v : order) {
271         if (sg[v].type == GateType::PI) {
272             dp[v] = 0;
273             continue;
274         }
275
276         for (int i = 0; i < (int)patterns.size(); i++) {
277             ll boundary_cost = matchCost(v, patterns[i].root, sg, patterns[i].g, dp, isInserted);
278
279             if (boundary_cost >= INF) continue;
280
281             ll total_cost = patterns[i].cost + boundary_cost;
282
283             if (total_cost < dp[v]) {
284                 dp[v] = total_cost;
285                 choose[v] = i;
286             }
287         }
288     }
289
290     ll ans = 0;
291     for (int out : original_outputs) {
292         ans += dp[out];
293     }
294
295     cout << "Minimum total cost = " << ans << endl;
296
297     cout << endl;
298     cout << "[Original nodes]" << endl;
299     for (int i = 0; i < ns; i++) {
300         cout << "node " << i << " (" << gate2str(sg[i].type) << "): ";
301
302         if (sg[i].type == GateType::PI) {
303             cout << "PI, cost = 0";
304         } else {
305             cout << "cost = " << dp[i];
306
307             if (choose[i] != -1) {
308                 cout << ", cell = " << patterns[choose[i]].name;
309             } else {
310                 cout << ", cell = NONE";
311             }
312         }
313
314         bool is_output = false;
315         for (int out : original_outputs) {
316             if (out == i) is_output = true;
317         }
318         if (is_output) cout << " [OUTPUT]";
319
320         cout << endl;
321     }
322
323     cout << endl;
324     cout << "[Inserted inverter-pair nodes]" << endl;
325
326     for (int i = ns; i < nsg; i++) {
327         cout << "node " << i << " (" << gate2str(sg[i].type) << "): ";
328         cout << "cost = " << dp[i];
329
330         if (choose[i] != -1) {
331             cout << ", cell = " << patterns[choose[i]].name;
332         } else {
333             cout << ", cell = NONE";
334         }
335
336         cout << endl;
337     }
338
339     return 0;
340 }
341

```

8. (c) Appendix: input.txt

```
1 18 17
2 PI PI PI PI PI PI PI PI NAND INV NAND NAND INV NAND NAND INV NAND
3 0 8
4 1 8
5 2 9
6 8 10
7 9 10
8 3 11
9 4 11
10 5 12
11 11 13
12 12 13
13 10 14
14 13 14
15 6 15
16 7 15
17 15 16
18 14 17
19 16 17
20 10
21 inv 1
22 2 1 1
23 PI INV
24 0 1
25 buf 0
26 3 2 2
27 PI INV INV
28 0 1
29 1 2
30 nand2 2
31 3 2 2
32 PI PI NAND
33 0 2
34 1 2
35 and2 3
36 4 3 3
37 PI PI NAND INV
38 0 2
39 1 2
40 2 3
41 nand3 3
42 6 5 5
43 PI PI PI NAND INV NAND
44 0 3
45 1 3
46 3 4
47 4 5
48 2 5
49 or2 4
50 5 4 4
51 PI PI INV INV NAND
52 0 2
53 1 3
54 2 4
55 3 4
56 ao121 3
57 7 6 6
58 PI PI PI NAND INV NAND INV
59 0 3
60 1 3
61 2 4
62 3 5
63 4 5
64 5 6
65 oai21 3
66 7 6 6
67 PI PI PI INV INV NAND NAND
68 0 3
69 1 4
70 3 5
71 4 5
72 5 6
73 2 6
74 ao122 4
75 8 7 7
76 PI PI PI PI NAND NAND NAND INV
77 0 4
78 1 4
79 2 5
80 3 5
81 4 6
82 5 6
83 6 7
84 nand4 4
85 9 8 8
86 PI PI PI PI NAND INV NAND INV NAND
87 0 4
88 1 4
89 4 5
90 5 6
91 2 6
92 6 7
93 7 8
94 3 8
```

8. (d) Appendix: output.txt

txt

```

1 Minimum total cost = 15
2 node 0: PI, cost = 0
3 node 1: PI, cost = 0
4 node 2: PI, cost = 0
5 node 3: PI, cost = 0
6 node 4: PI, cost = 0
7 node 5: PI, cost = 0
8 node 6: PI, cost = 0
9 node 7: PI, cost = 0
10 node 8: cost = 2, cell = nand2
11 node 9: cost = 1, cell = inv
12 node 10: cost = 5, cell = nand2
13 node 11: cost = 2, cell = nand2
14 node 12: cost = 1, cell = inv
15 node 13: cost = 5, cell = nand2
16 node 14: cost = 12, cell = nand2
17 node 15: cost = 2, cell = nand2
18 node 16: cost = 3, cell = inv
19 node 17: cost = 15, cell = nand3
20

```

8. (e) Appendix: output_improve.txt

txt

```

1 Minimum total cost = 12
2
3 [Original nodes]
4 node 0 (PI): PI, cost = 0
5 node 1 (PI): PI, cost = 0
6 node 2 (PI): PI, cost = 0
7 node 3 (PI): PI, cost = 0
8 node 4 (PI): PI, cost = 0
9 node 5 (PI): PI, cost = 0
10 node 6 (PI): PI, cost = 0
11 node 7 (PI): PI, cost = 0
12 node 8 (NAND): cost = 2, cell = nand2
13 node 9 (INV): cost = 1, cell = inv
14 node 10 (NAND): cost = 5, cell = nand2
15 node 11 (NAND): cost = 2, cell = nand2
16 node 12 (INV): cost = 1, cell = inv
17 node 13 (NAND): cost = 5, cell = nand2
18 node 14 (NAND): cost = 10, cell = nand2
19 node 15 (NAND): cost = 2, cell = nand2
20 node 16 (INV): cost = 3, cell = inv
21 node 17 (NAND): cost = 12, cell = oai21 [OUTPUT]
22
23 [Inserted inverter-pair nodes]
24 node 18 (INV): cost = 1, cell = inv
25 node 19 (INV): cost = 0, cell = buf
26 node 20 (INV): cost = 1, cell = inv
27 node 21 (INV): cost = 0, cell = buf
28 node 22 (INV): cost = 1, cell = inv
29 node 23 (INV): cost = 0, cell = buf
30 node 24 (INV): cost = 3, cell = inv
31 node 25 (INV): cost = 2, cell = buf
32 node 26 (INV): cost = 0, cell = buf
33 node 27 (INV): cost = 1, cell = inv
34 node 28 (INV): cost = 1, cell = inv
35 node 29 (INV): cost = 0, cell = buf
36 node 30 (INV): cost = 1, cell = inv
37 node 31 (INV): cost = 0, cell = buf
38 node 32 (INV): cost = 1, cell = inv
39 node 33 (INV): cost = 0, cell = buf
40 node 34 (INV): cost = 3, cell = inv
41 node 35 (INV): cost = 2, cell = buf
42 node 36 (INV): cost = 0, cell = buf
43 node 37 (INV): cost = 1, cell = inv
44 node 38 (INV): cost = 3, cell = aoi21
45 node 39 (INV): cost = 4, cell = inv
46 node 40 (INV): cost = 3, cell = aoi21
47 node 41 (INV): cost = 4, cell = inv
48 node 42 (INV): cost = 1, cell = inv
49 node 43 (INV): cost = 0, cell = buf
50 node 44 (INV): cost = 1, cell = inv
51 node 45 (INV): cost = 0, cell = buf
52 node 46 (INV): cost = 3, cell = inv
53 node 47 (INV): cost = 2, cell = buf
54 node 48 (INV): cost = 9, cell = aoi21
55 node 49 (INV): cost = 10, cell = inv
56 node 50 (INV): cost = 2, cell = buf
57 node 51 (INV): cost = 3, cell = inv
58

```